Design Patterns in GO

Index

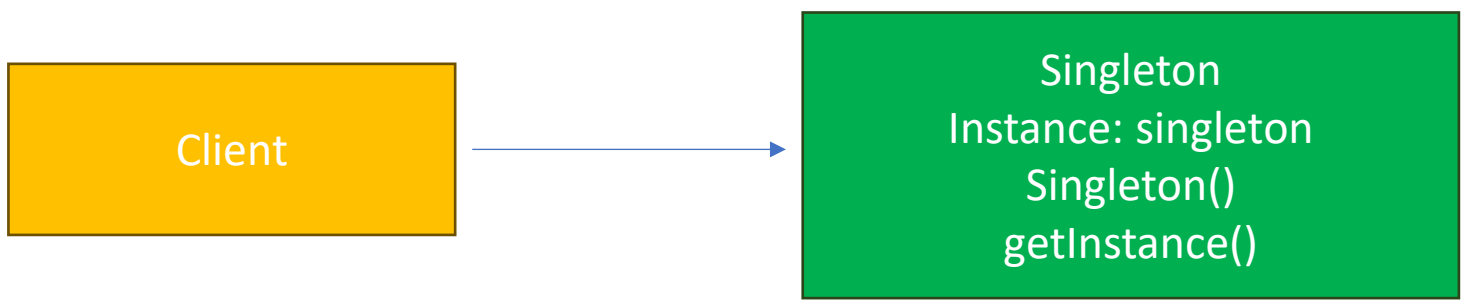
Creational Patterns

- Singleton
- Builder
- Factory method
- Abstract factory

Singleton Pattern

- It is a creational pattern from which objects can be created only once and provides a global access point to it.

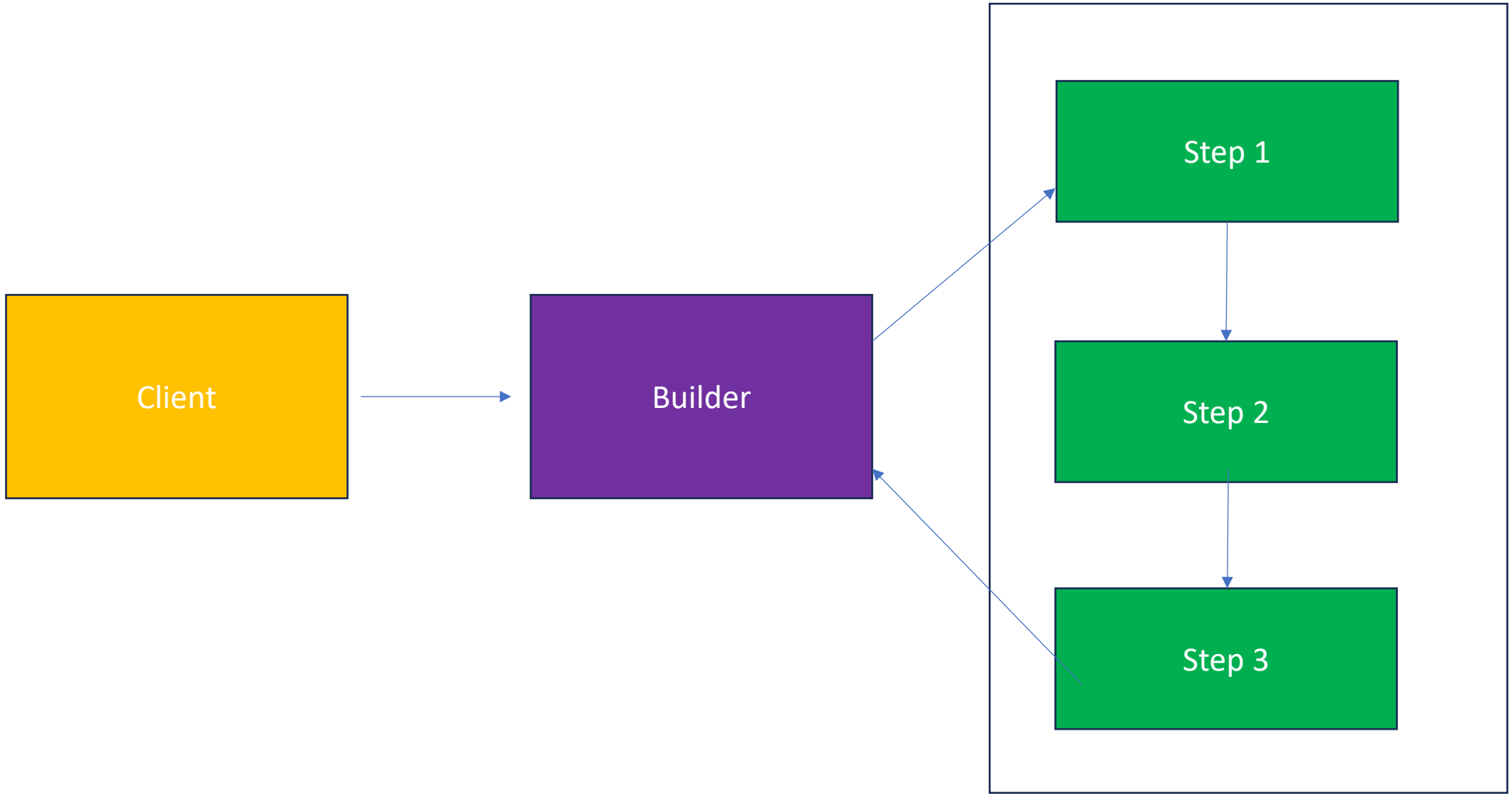
- For database ,logger , object cache etc initiations. For systemwide access


Client
Singleton
Instance: singleton
Singleton()
getInstance()

Builder

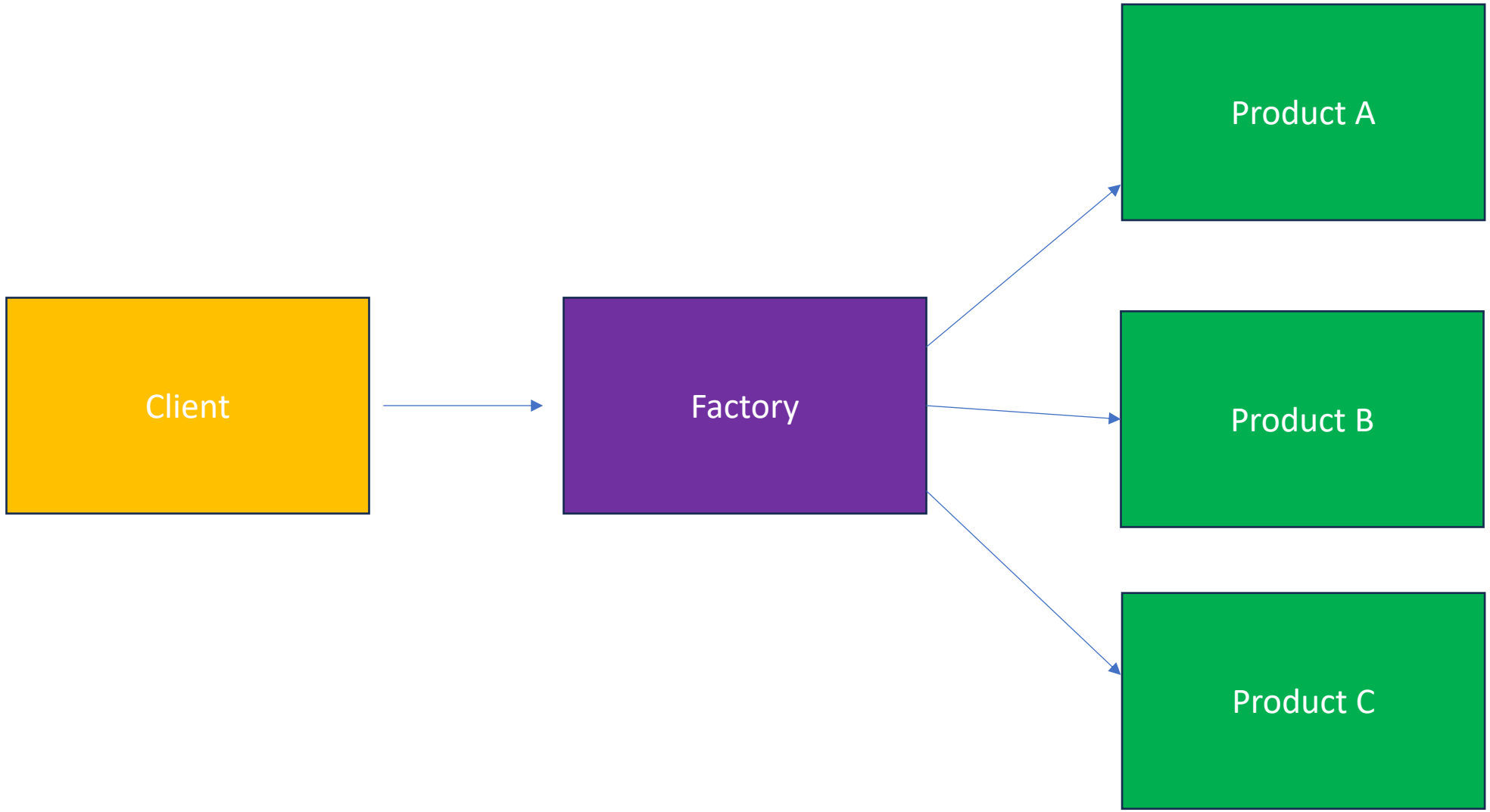
- It is a creational pattern that provides interface for creating object in a superclass, but allows subclasses to alter the type of objects that will be created
- Multi difference configuration of objects can be crated with this pattern in combination of different steps.
- It provides a clean way to create complex objects step by step

- Like sql query builder, other complex builder


Client
Builder
Step 1
Step 2
Step 3

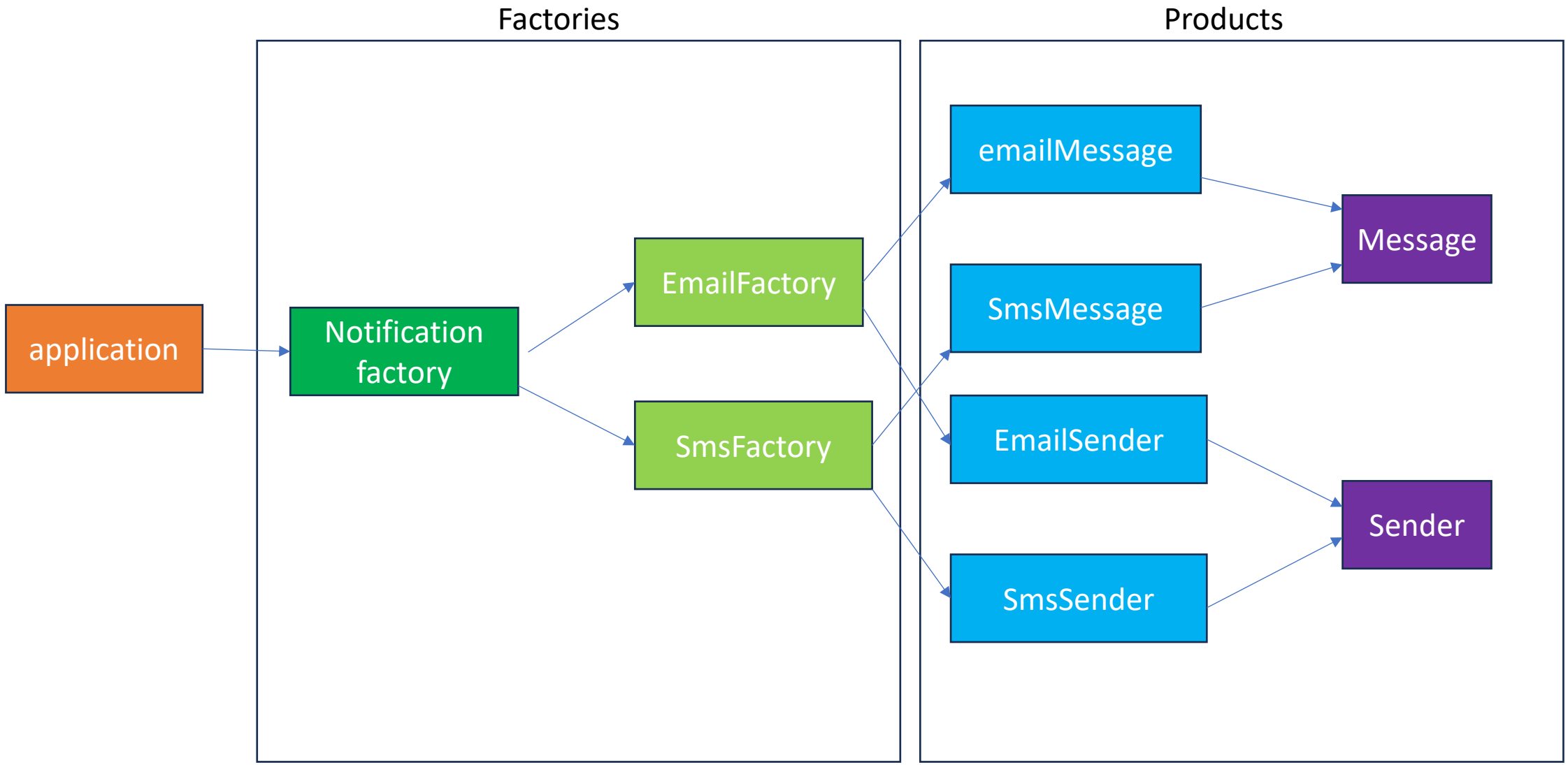
factory

- It is a creational pattern that provides an interface for creating objects in superclass but allows subclass to decide which type of object to instantiate.
- Like notification service. The interface contract has the methods and all the classes implementing it can fulfill contract in their own way .like for SMS, email, push notification. It reduces complex and repetitive object creation


Client
Factory
Product A
Product B
Product C

Abstract factory

- It provides a interface for creating a family of related or dependent objects without specifying their concrete classes .it is factory of factories
- Like creating all theme light/dark components .
- Gui or OS specific components

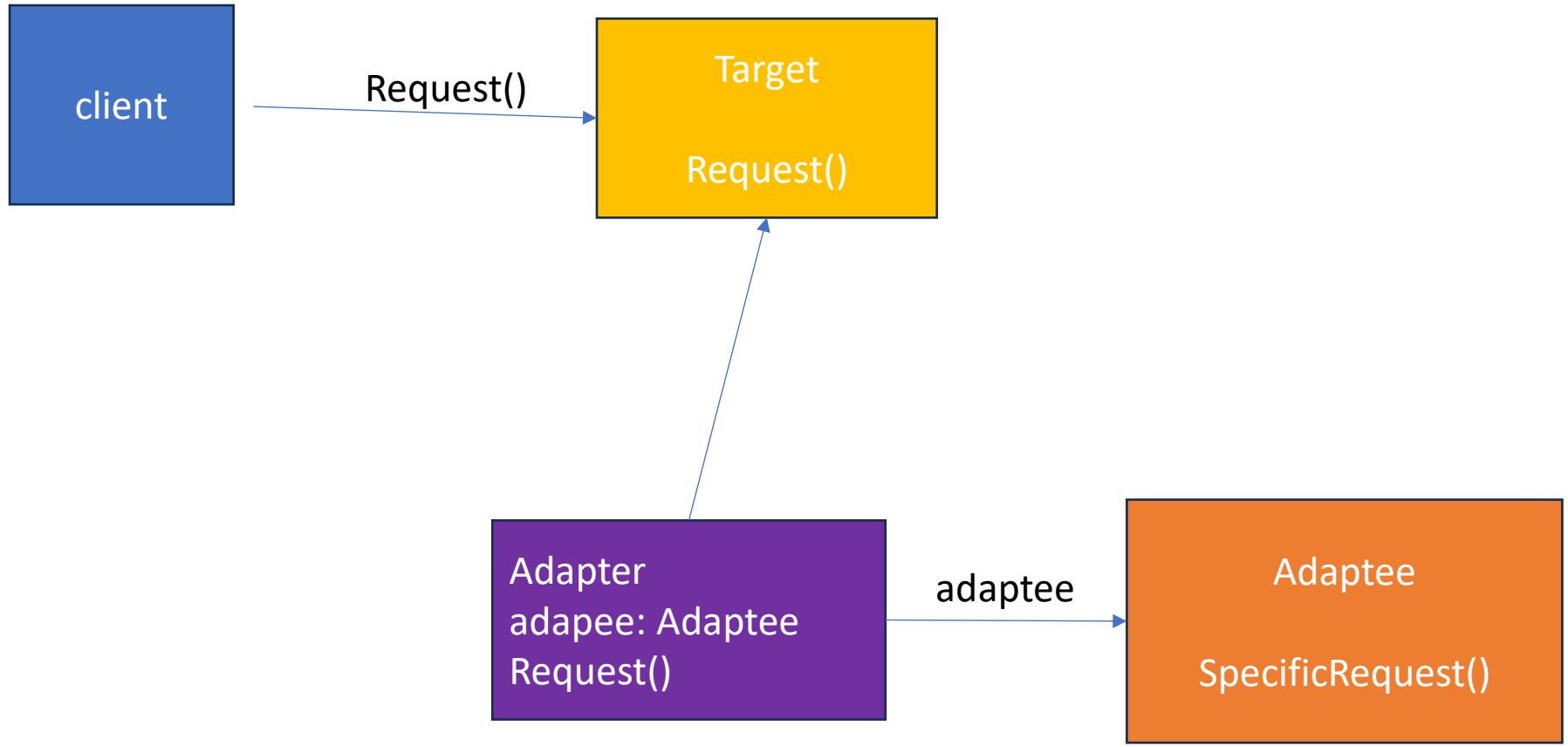

Factories
Products
application
Notification factory
EmailFactory
SmsFactory
emailMessage
SmsMessage
EmailSender
SmsSender
Message
Sender

Structural Pattern

- Adapter
- Façade
- Decorator
- composite

Adapter Pattern

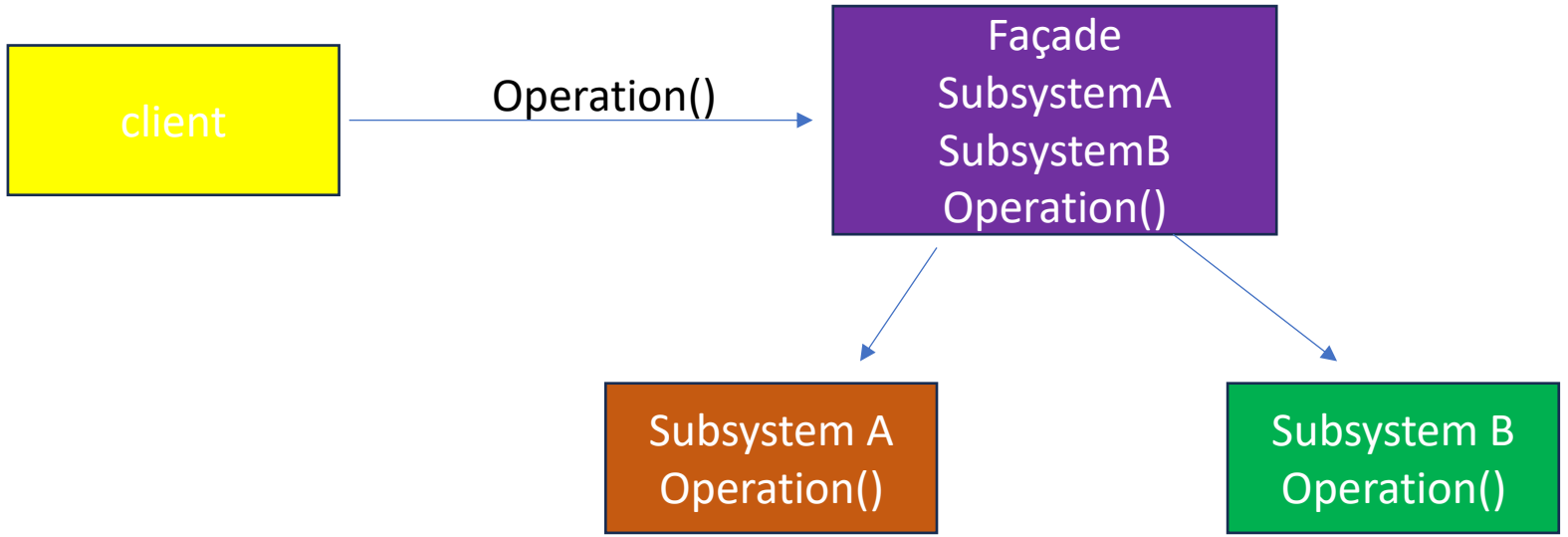
- Adapter acts as a bridge between 2 incompatible interfaces allowing them to work together . Used for third party library or legacy code integration into new systems.
- It enables incompatible classes to work without change.
- Like input is xml for a class but response got is Json so we convert json to xml in adapter and feed to legacy class


client
Request()
Target
Request()
Adapter
adapee: Adaptee
Request()
adaptee
Adaptee
SpecificRequest()

Facade Pattern

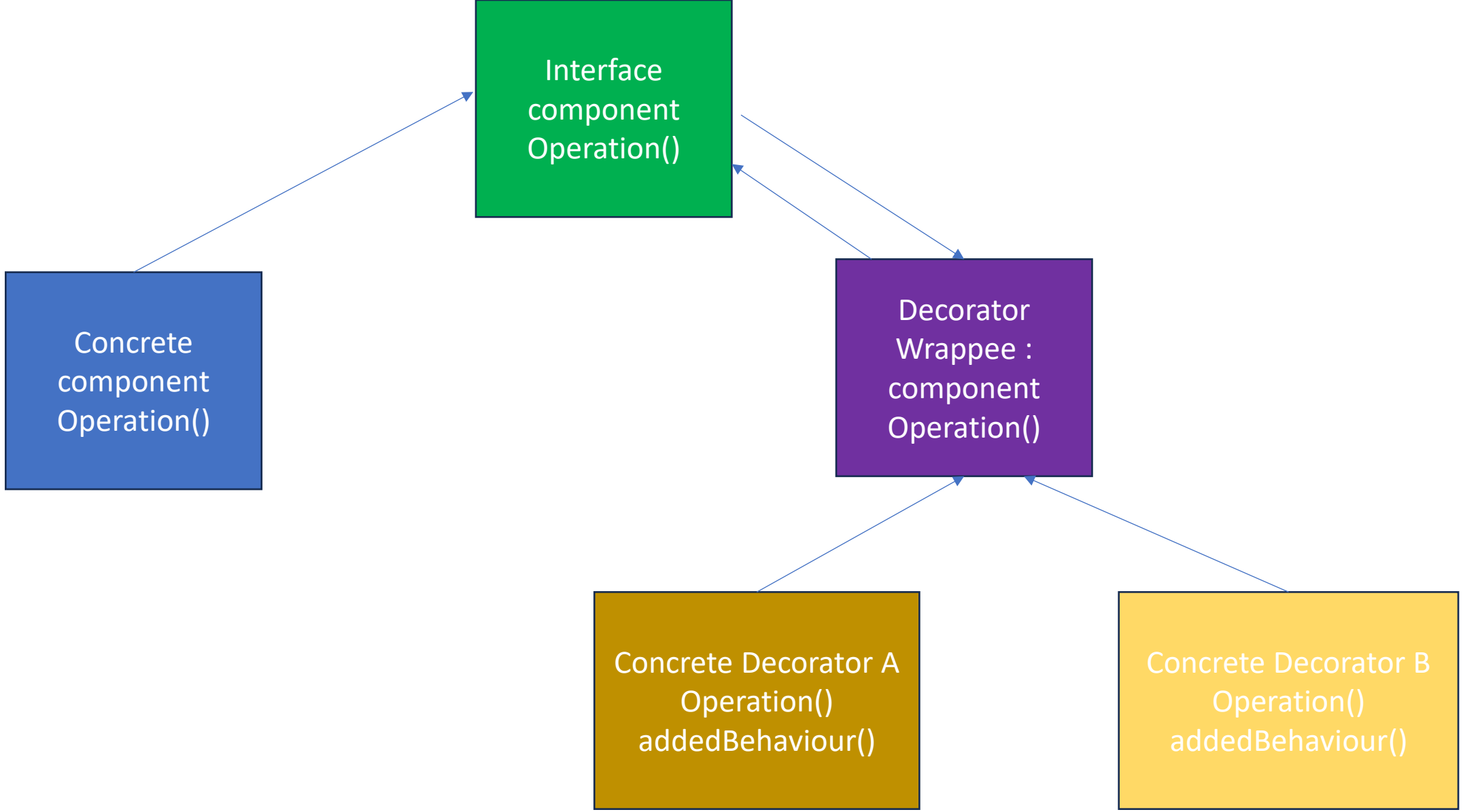
- Facade pattern provides simplified interface to a complex set of class, library or framework. It wraps complex interactions behind a clean interface.
- Client interact with 1 object while facade coordinate with subsystem calls behind.

- Examples : home automation system, banking system etc any system with multiple object , multistep function interaction


client
Operation()
Façade
SubsystemA
SubsystemB
Operation()
Subsystem A
Operation()
Subsystem B
Operation()

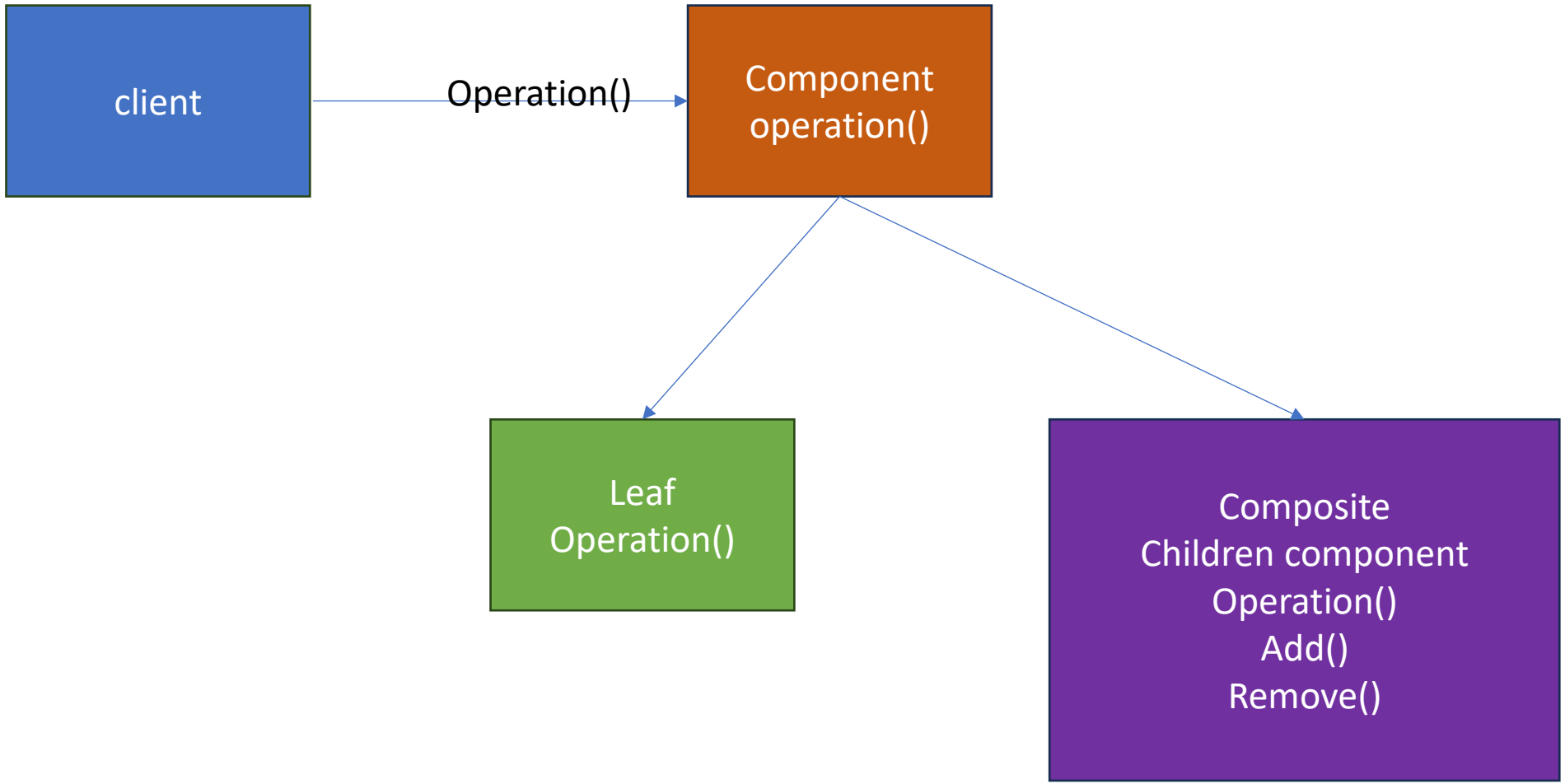
Decorator Pattern

- It lets dynamically add new responsibilities to object without changing underlying code.
- It can extend functionality without subclassing, compose behavior at runtime,


Interface
component
Operation()
Concrete
component
Operation()
Decorator
Wrappee :
component
Operation()
Concrete Decorator A
Operation()
addedBehaviour()
Concrete Decorator B
Operation()
addedBehaviour()

Composite Pattern

- Structural pattern that organizes object into tree like structure .to treat individual and composite objects alike through common interface. Simplifies extension
- Like filesystem, UI hierarchy, html , organizational charts etc
- When you want to perform operation on leaf and composite node both and avoid writing special logic

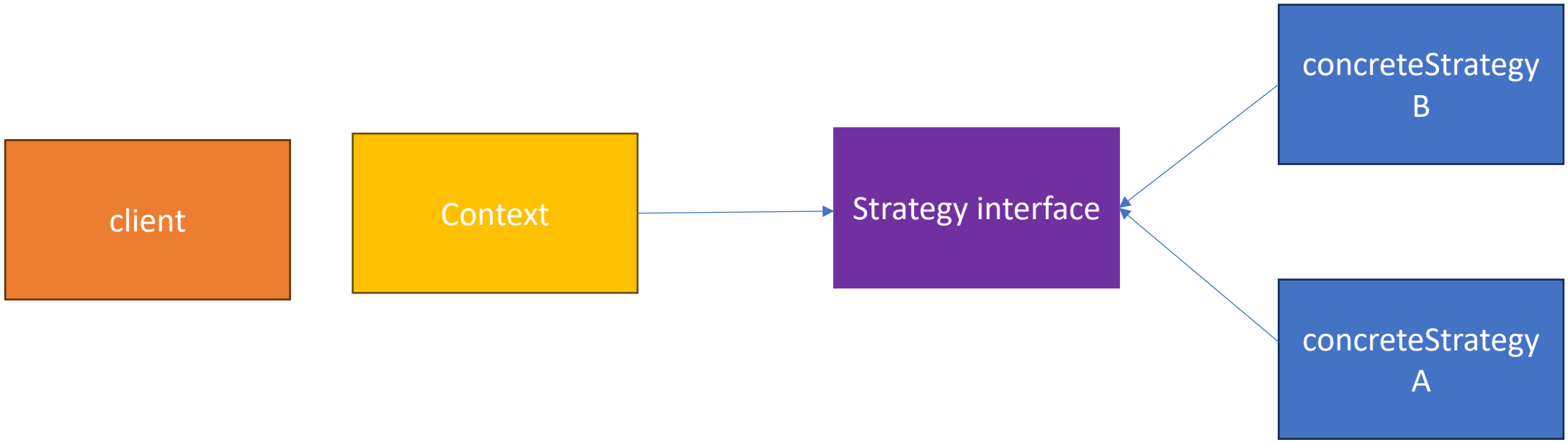

client
Operation()
Component
operation()
Leaf
Operation()
Composite
Children component
Operation()
Add()
Remove()

Behavioral Patterns

- Strategy
- Observer
- State
- Command
- mediator

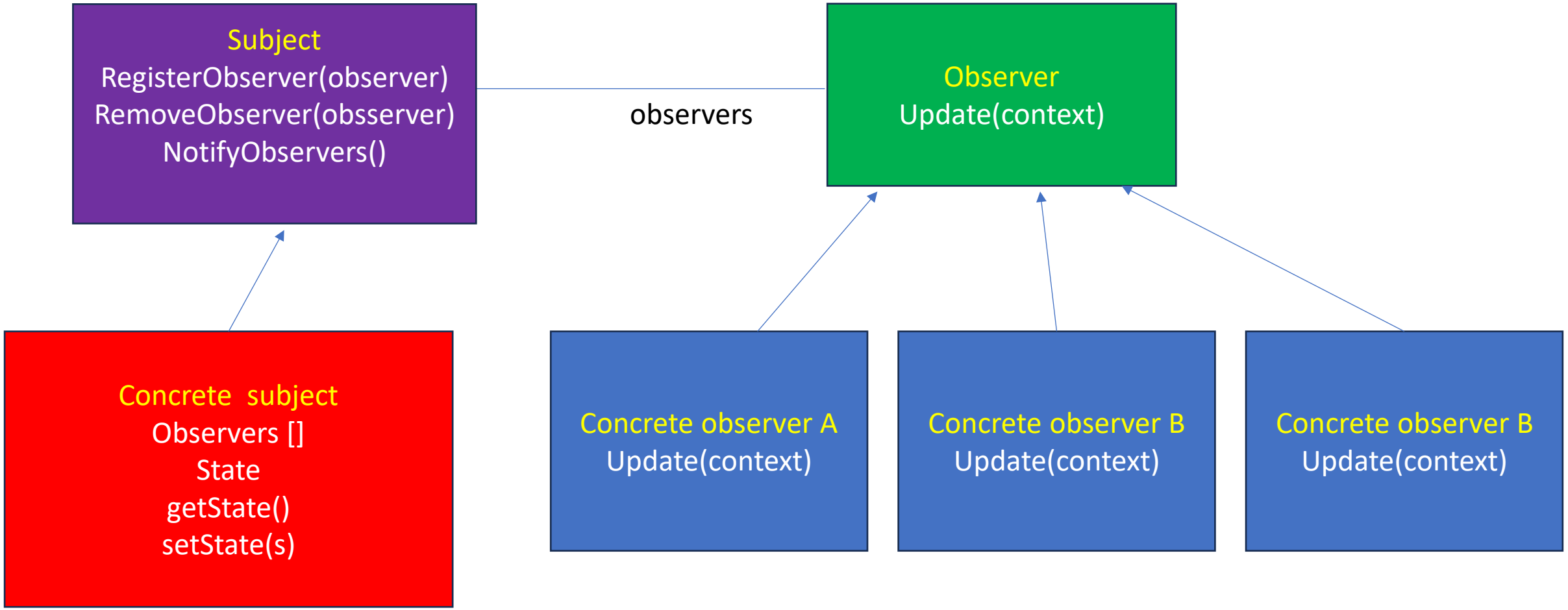
Strategy Pattern

- Makes algorithm injectable and individual to class/ struct that uses it.
- So it can be changed at runtime.
- Example for a payment strategy user can pay with credit card or upi or any other way, this payment structs follows same interface contract so they can be implemented and changed when needed.


client
Context
Strategy interface
concreteStrategy B
concreteStrategy A

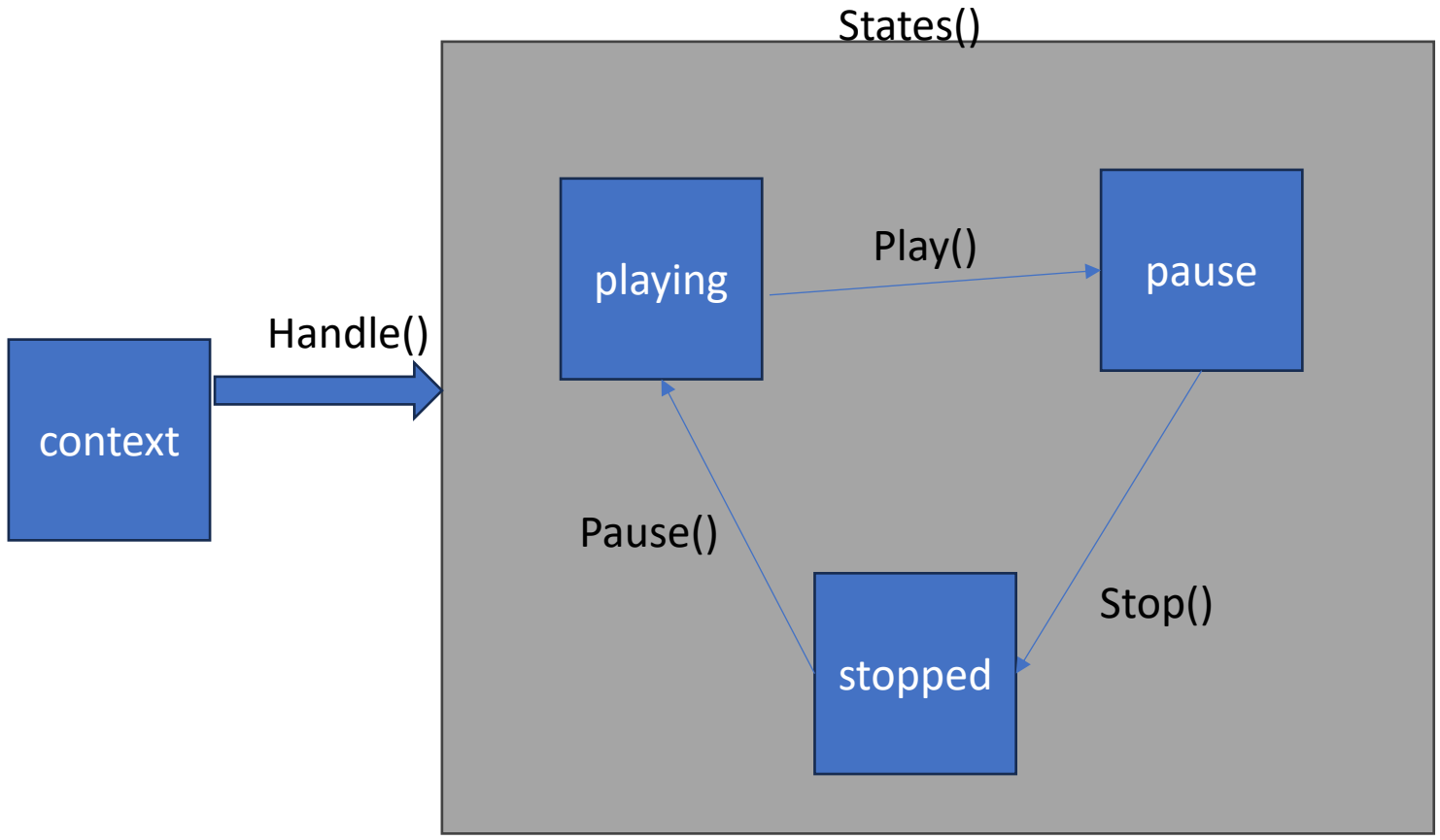
Observer Pattern

- This is used to send change state notification to all its listeners .
- Creates a one to many dependency btw object and dependents, when subject changes all dependents are notified. [Broadcasting changes]
- If a person posts something then all the friends are notified for new post.
- In stock market or gui listeners


Subject
RegisterObserver(observer)
RemoveObserver(obsserver)
NotifyObservers()
observers
Observer
Update(context)
Concrete subject
Observers []
State
getState()
setState(s)
Concrete observer A
Update(context)
Concrete observer B
Update(context)
Concrete observer B
Update(context)

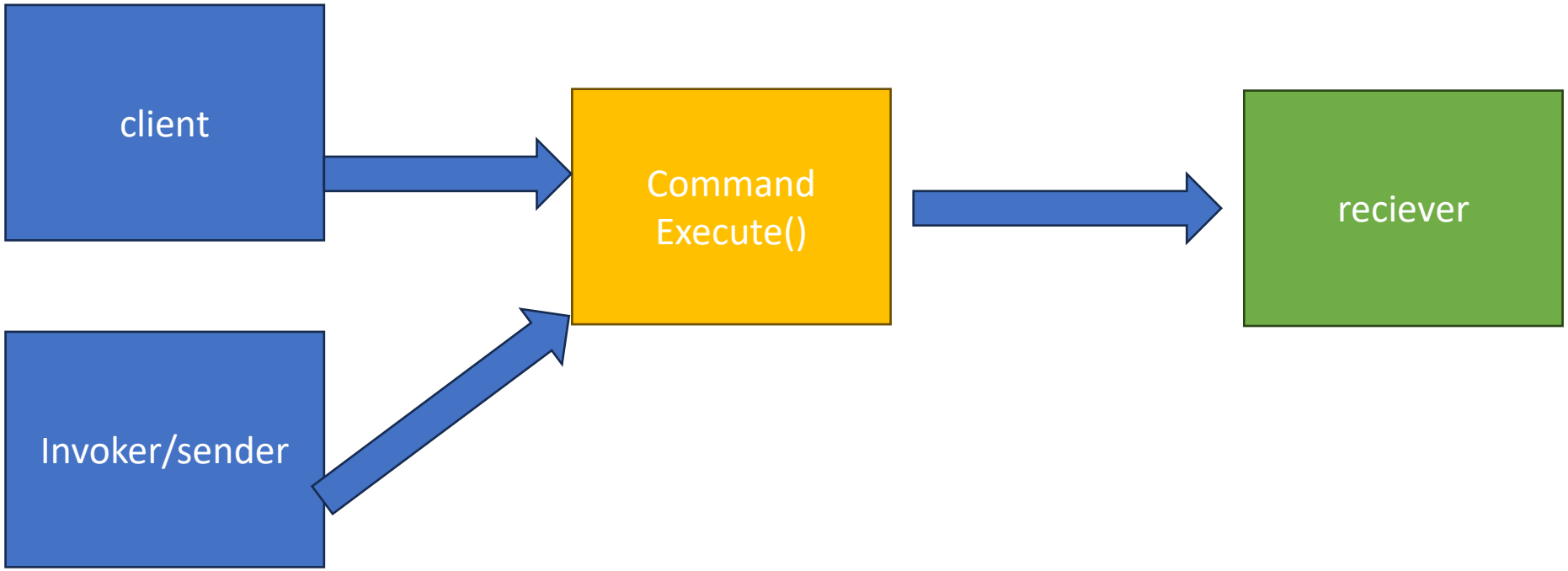
State Pattern

- Is a pattern that lets object alter its behavior when its internal state changes. It encapsulates state specific behavior in separate state classes , allowing object to manage transition cleanly
- It prevents a lot of if else switch ,makes code clean and predictable
- All behavior are in separate state specific classes with relatable behavior
- like a traffic light with specific state red, yellow ,green with their specific timeout
- Or like a vending machine ,with item selected, money added, item dispensed state for operations


States()
playing
Play()
pause
context
Handle()
Pause()
stopped
Stop()

Command Pattern

- Converts request into objects to decouple sender and receiver .
- Like a waiter takes a order on a paper slip and sends to chef, the paper works like a command with all information for chef to take action to cook meal
- Second example :
- Like a remote button by user to turn on tv
- Here remote is invoker and button is command object with message
- And tv device is the receiver
- When button pressed command.execute() is called which triggers action on receiver
- Used in gui buttons, undoredo in text editors,storing command history
- Queue or delay or log requests

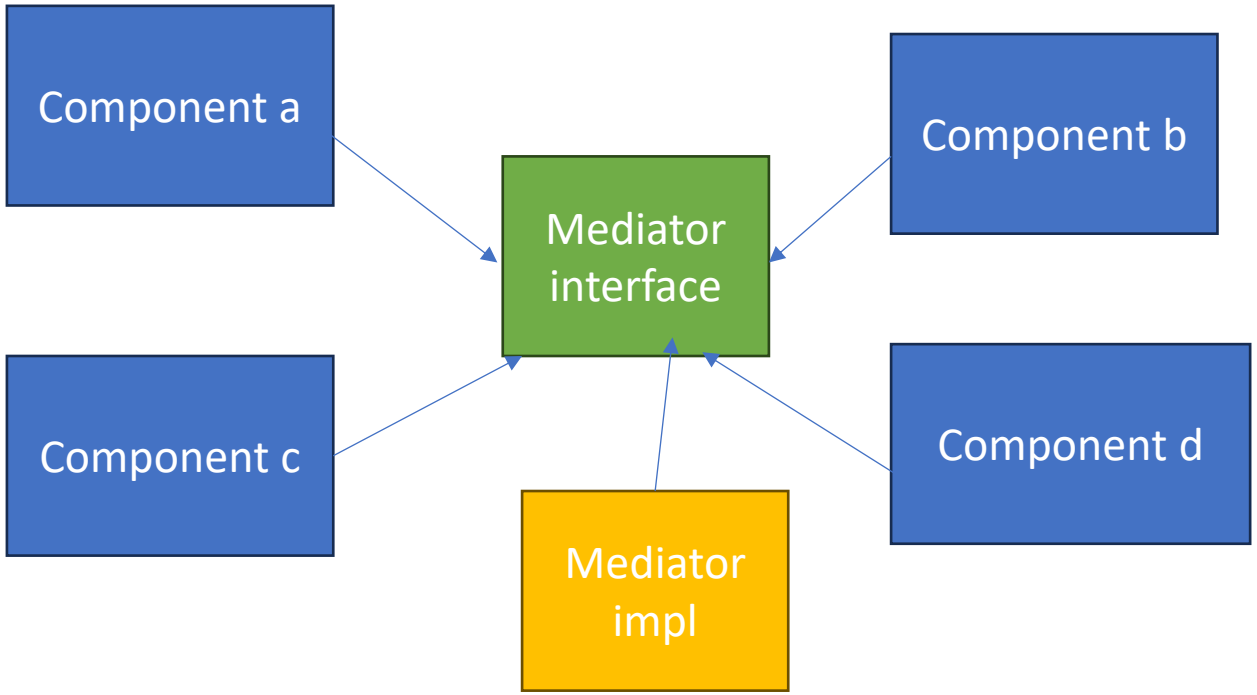

client
Command
Execute()
reciever
Invoker/sender

Mediator Pattern

- This restricts direct communication between objects. Also like a hub spoke. So all communication goes through it to reduce chaotic dependencies between objects

- Like planes communicate to traffic controller rather than to each other . And controller convey message so they are only aware of traffic controller for their message.

mediator implementation



others

- Null object
- MVC
- Repository
- Game loop
- Thread pool
- Producer consumer

Repository Link

- https://github.com/subpxl/engineering-deep-dive

- Thank You